

《程序设计艺术与方法》课程实验报告

实验名称	实验二 搜索算法的实现						
姓 名	李泽浩	学院	计算机与信息学院	班 级	软件工程25-9 班	学 号	2025218895
实验日期			指导教师	曹力		成 绩	
一、实验目的和要求 1. 掌握宽度优先搜索（BFS）算法。 2. 掌握深度优先搜索（DFS）算法。							
二、实验预习内容 1. 预习实验指导书中的搜索算法部分内容，理解状态空间树的概念。 2. 了解深度优先搜索（DFS）的递归与回溯思想，以及广度优先搜索（BFS）利用队列进行层次遍历的原理。 3. 预习经典问题（如走迷宫、八皇后、骑士游历、倒水问题）的数学模型与状态转移规则。							
三、实验项目摘要 1. 走迷宫代码运行：利用搜索算法（DFS 或 BFS）探索从起点到终点的路径，体会搜索的回溯或层次扩展思想。 2. 八皇后问题：在一个国际象棋棋盘上放八个皇后，使得任何两个皇后之间不相互攻击，使用深度优先搜索与回溯法求出所有的布棋方法。并思考推广到 N 皇后时的算法优化。 3. 骑士游历问题：在国际棋盘上使一个骑士遍历所有的格子一遍且仅一遍。利用深度优先搜索结合启发式规则（如 Warnsdorff 规则）输出一条给定起点的游历路径。 4. 倒水问题：给定 2 个没有刻度的容器（容积为 A 和 B），利用广度优先搜索求出如何装出 L 升水的最短步骤；若无法实现则输出 No Solution。							
四、实验结果与分析（源程序及相关说明） 1. 走迷宫问题（DFS 与 BFS 综合对比）：具体解法与说明 核心思想： 本实验在同一个迷宫地图上，分别使用深度优先搜索（DFS）和广度优先搜索（BFS）进行寻路。 DFS 采用“不撞南墙不回头”的策略，适合探索所有可行解； BFS 采用“水波纹式层层扩展”的策略，能够保证找到的解是最短路径。 具体解法与实现细节： 地图与方向定义：使用 5x5 二维数组 maze 表示迷宫，0 为路，1 为墙。定义 dx 和 dy 数组用于控制上下左右四个探索方向。							

DFS 解法（递归与回溯）：

利用系统调用栈实现递归探索。从起点出发，每次选择一个可走且未访问的相邻方向深入。

利用全局数组 `visited` 记录访问状态。走过的点压入 `path1` 容器。

若四个方向都走不通，则触发回溯：将当前点从 `path1` 弹出，并在 `visited` 中重置为未访问，退回上一格继续尝试。

BFS 解法（队列与前驱记录）：

独立状态：为防止干扰，BFS 使用了独立的局部访问数组 `bfsVisited`。

队列扩展：利用 STL 的 `queue` 存储待访问坐标。起点入队后，每次取出队首元素，将其周围合法且未访问的邻居节点标记并全部入队。

路径反推：BFS 虽然能最快找到终点，但探索过程是发散的。为了输出确切的最短路径，定义了二维数组 `prev[ROW][COL]` 来记录每个节点的“前驱节点”。

生成最短路径：当队首元素到达终点时，利用 `prev` 数组从终点不断倒推回起点，将坐标存入 `path2`。

最后使用 `algorithm` 库中的 `reverse` 函数将 `path2` 倒置，即得到从起点到终点的正确正向最短路径。

```
#include <iostream>
#include <vector>
#include <queue>
#include <cstring>
#include <algorithm>
using namespace std;

const int ROW = 5;
const int COL = 5;
// 迷宫地图，0 表示通，1 表示墙
int maze[ROW][COL] = {
    {0, 1, 0, 0, 0},
    {0, 1, 0, 1, 0},
    {0, 0, 0, 0, 0},
    {0, 1, 1, 1, 0},
    {0, 0, 0, 1, 0}
};

int visited[ROW][COL] = {0};
// 方向数组：上下左右
int dx[4] = {-1, 1, 0, 0};
int dy[4] = {0, 0, -1, 1};
struct Point { int x, y; };
vector<Point> path1, path2;
// ===== DFS =====
bool dfs(int x, int y) {
    if (x == ROW - 1 && y == COL - 1) {
        path1.push_back({x, y});
        return true;
    }
    visited[x][y] = 1;
    path1.push_back({x, y});
```

```

    for (int i = 0; i < 4; i++) {
        int nx = x + dx[i];
        int ny = y + dy[i];
        if (nx >= 0 && nx < ROW && ny >= 0 && ny < COL
            && maze[nx][ny] == 0 && !visited[nx][ny]) {
            if (dfs(nx, ny)) return true;
        }
    }
    // 回溯
    path1.pop_back();
    visited[x][y] = 0;
    return false;
}

// ===== BFS =====
bool bfs(int startX, int startY) {
    // BFS 用自己的 visited 数组，避免与 DFS 冲突
    int bfsVisited[ROW][COL] = {0};
    Point prev[ROW][COL]; // 记录每个点的前驱，用于反推路径
    queue<Point> q;
    q.push({startX, startY});
    bfsVisited[startX][startY] = 1;
    prev[startX][startY] = {-1, -1}; // 起点标记无前驱
    while (!q.empty()) {
        Point cur = q.front();
        q.pop();
        if (cur.x == ROW - 1 && cur.y == COL - 1) {
            // 反推路径，从终点回溯到起点
            Point p = cur;
            while (!(p.x == -1 && p.y == -1)) {
                path2.push_back(p);
                p = prev[p.x][p.y];
            }
            // 反转得到从起点到终点的顺序
            reverse(path2.begin(), path2.end());
            return true;
        }
        for (int i = 0; i < 4; i++) {
            int nx = cur.x + dx[i];
            int ny = cur.y + dy[i];
            if (nx >= 0 && nx < ROW && ny >= 0 && ny < COL
                && maze[nx][ny] == 0 && !bfsVisited[nx][ny]) {
                bfsVisited[nx][ny] = 1;
                prev[nx][ny] = cur;
                q.push({nx, ny});
            }
        }
    }
}

```

```

    }
}
return false;
}
int main() {
    cout << "===== 走迷宫问题 =====" << endl;
    // DFS
    if (dfs(0, 0)) {
        cout << "DFS 找到一条路径: " << endl;
        for (auto p : path1) {
            cout << "(" << p.x << ", " << p.y << ") -> ";
        }
        cout << "终点" << endl;
    } else {
        cout << "DFS 无路可走!" << endl;
    }
    // BFS (visited 独立, 不受 DFS 影响)
    if (bfs(0, 0)) {
        cout << "BFS 找到最短路径!" << endl;
        for (auto p : path2) {
            cout << "(" << p.x << ", " << p.y << ") -> ";
        }
        cout << "终点" << endl;
    } else {
        cout << "BFS 无路可走!" << endl;
    }
    return 0;
}

```

2. 八皇后问题：具体解法与说明

核心算法：深度优先搜索（DFS）与空间状态剪枝

状态表示：由于同一行不可能放置两个皇后，因此不需要二维数组记录棋盘。使用一个一维数组 `queen_pos`，其索引代表“行号”，对应的值代表该行皇后所在的“列号”。

检查是否能放置：

程序通过三个布尔数组来实现 $O(1)$ 复杂度的冲突检测：

`col_used[col]`：记录第 `col` 列是否已经有皇后。

`diag1_used[row + col]`：记录主对角线（左下到右上）是否被占用。在同一条主对角线上的格子，其“行号+列号”是一个常数。

`diag2_used[row - col + N - 1]`：记录副对角线（左上到右下）是否被占用。在同一条副对角线上的格子，其“行号-列号”是一个常数（加上 $N-1$ 是为了防止数组越界出现负数下标）。

具体解法：

按行推进（`row` 从 0 到 $N-1$ ）。在当前行，尝试遍历所有的列（0 到 $N-1$ ）。如果当前列 `col` 以及

对应的两条对角线在布尔数组中都为 `false`（未被占用），则说明该位置安全。将该位置标记为占用，并递归调用下一行 `solveNQueens(row + 1)`。当下一行的递归返回后，将当前位置的三个布尔数组状态全部恢复为 `false`（回溯），以便尝试当前行的下一个合法列。

```
#include <iostream>
#include <vector>

using namespace std;
int N = 8;
int total_solutions = 0;
vector<int> queen_pos; // 记录每行皇后的列号
vector<bool> col_used, diag1_used, diag2_used; // 记录列和对角线是否被占用
void printSolution() {
    // 这里只打印前 2 种解的具体布局，但统计所有解的数量
    if (total_solutions <= 2) {
        cout << "第 " << total_solutions << " 种解: " << endl;
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++) {
                if (queen_pos[i] == j) cout << "Q ";
                else cout << ". ";
            }
            cout << endl;
        }
        cout << endl;
    }
}

void solveNQueens(int row) {
    if (row == N) {
        total_solutions++; // 找到一个解，统计数量并打印前 2 种解
        printSolution();
        return;
    }
    for (int col = 0; col < N; col++) {
        // diag1: row + col; diag2: row - col + N - 1
        if (!col_used[col] && !diag1_used[row + col] && !diag2_used[row - col + N - 1]) {
            // 放置皇后
            queen_pos[row] = col;
            col_used[col] = diag1_used[row + col] = diag2_used[row - col + N - 1] = true;

            solveNQueens(row + 1); // 递归到下一行

            // 回溯，撤销放置
            col_used[col] = diag1_used[row + col] = diag2_used[row - col + N - 1] = false;
        }
    }
}
```

```

}

int main() {
    cout << "===== 八皇后问题 =====" << endl;
    N = 8;
    queen_pos.assign(N, 0);
    col_used.assign(N, false); // 记录列是否被占用
    diag1_used.assign(2 * N - 1, false); // 记录主对角线是否被占用
    diag2_used.assign(2 * N - 1, false); // 记录副对角线是否被占用

    solveNQueens(0);
    cout << N << " 皇后问题共有 " << total_solutions << " 种解法。" << endl;
    return 0;
}

```

3. 骑士游历问题：具体解法与说明

核心算法：深度优先搜索（DFS） + Warnsdorff 启发式规则优化

问题难点：

国际象棋棋盘有 64 个格子，骑士走日字型，每个格子最多有 8 种走法。纯粹的 DFS 回溯会导致状态空间爆炸（时间复杂度呈指数级增长），程序极大概率会陷入“死胡同”长时间无法得出结果。

具体解法（Warnsdorff 规则）：

必须引入贪心启发式策略来“剪枝”。Warnsdorff 规则的核心思想即为优先选择下一步可跳位置最少的格子。

计算度数：定义一个函数 `getDegree(x, y)`。对于棋盘上的某个合法位置，计算从它出发，还能跳到多少个尚未被访问的合法格子（即计算“出度”）。

排序候选节点：在 DFS 的每一步，找出当前骑士能跳到的所有合法下一格。不再按固定的上下左右顺序去试，而是计算这些合法下一格各自的“出度”。

将这些候选格子按照“出度”从大到小进行升序排序。

贪心深入：优先跳向那个“出度”最小的格子。这样做可以迫使骑士先去走棋盘边缘和角落这些“容易走死”的格子，把广阔的中心区域留到最后，从而极大地减少了错误路径的回溯次数，通常能在线性时间内迅速找到一条遍历 64 格的解。

```

#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;
const int SIZE = 8;
int board[SIZE][SIZE];
// 骑士走日字的 8 个方向
int cx[8] = {1, 1, 2, 2, -1, -1, -2, -2};

```

```

int cy[8] = {2, -2, 1, -1, 2, -2, 1, -1};
bool isSafe(int x, int y) {
    return (x >= 0 && x < SIZE && y >= 0 && y < SIZE && board[x][y] == -1);
}
// 计算某位置下一步的可走位置数量 (Warnsdorff 规则的核心)
int getDegree(int x, int y) {
    int count = 0;
    for (int i = 0; i < 8; ++i) {
        if (isSafe(x + cx[i], y + cy[i])) count++;
    }
    return count;
}
bool solveKnightTour(int x, int y, int step) {
    board[x][y] = step;
    if (step == SIZE * SIZE) return true; // 已遍历满 64 格
    // 收集所有可走的下一步
    vector<pair<int, int>> next_moves;
    for (int i = 0; i < 8; ++i) {
        int nx = x + cx[i];
        int ny = y + cy[i];
        if (isSafe(nx, ny)) {
            next_moves.push_back({getDegree(nx, ny), i}); // 存储 (度数, 方向索引)
        }
    }
    // 按照 Warnsdorff 规则, 根据下一步的可走度数升序排序
    sort(next_moves.begin(), next_moves.end());
    // 优先走度数最小的分支
    for (auto move : next_moves) {
        int dir = move.second;
        if (solveKnightTour(x + cx[dir], y + cy[dir], step + 1)) {
            return true;
        }
    }
    // 回溯
    board[x][y] = -1;
    return false;
}
int main() {
    cout << "=====骑士游历问题 =====< endl;
    for (int i = 0; i < SIZE; i++)
        for (int j = 0; j < SIZE; j++)
            board[i][j] = -1;
    int start_x = 0, start_y = 0; // 给定起始顶点
    if (solveKnightTour(start_x, start_y, 1)) {

```

```

        cout << "找到一条游历路径（数字代表走步顺序）：" << endl;
        for (int i = 0; i < SIZE; i++) {
            for (int j = 0; j < SIZE; j++) {
                printf("%2d ", board[i][j]);
            }
            cout << endl;
        }
    } else {
        cout << "无法完成游历" << endl;
    }
    return 0;
}

```

4. 倒水问题：具体解法与说明

核心算法：广度优先搜索（BFS）

算法选择依据：题目要求的是“如何只用两个瓶装出 L 升水”，暗示需要寻找最短的步骤。BFS 具有按层次展开的特性，最先找到的目标状态必定是步数最少的，因此 BFS 是求解此问题的最优解法。

状态设计：将两个水杯的当前水量看作一个系统状态，定义一个结构体包含：杯子 A 的水量、杯子 B 的水量、达到该状态执行的操作描述、以及父节点的索引（用于最后逆向推导操作步骤）。

具体解法：

初始化：将初始状态 (0, 0) 放入队列（本程序中使用 vector 模拟队列以保留历史节点），并在二维布尔数组 visited[A 容量+1][B 容量+1] 中将 (0, 0) 标记为已访问。

状态转移（分支扩展）：每次从队列头部取出一个状态，无论当前水杯里有多少水，下一步都严格只有 6 种可能的操作：

装满 A

装满 B

倒空 A

倒空 B

把 A 的水倒入 B（可能 A 倒空了，也可能 B 满了 A 还有剩）

把 B 的水倒入 A

去重与入队：对上述 6 种操作计算出新的水量。如果这个新的水量组合在 visited 数组中未被记录过，则标记为已访问，并将其作为新状态推入队列尾部，同时记录下父节点的索引号。

终止与回溯：一旦队列中产生的某个新状态满足 A 或 B 的水量等于目标值 L，搜索立刻终止。通过追踪该节点的 parent_id，一直倒推回起始节点，将路径反转后输出，即得到了最少步数的完整倒水操作流程。如果队列被清空仍未找到目标，则说明该水量无法被配出（No Solution）。

```

#include <iostream>
#include <vector>
#include <queue>
#include <string>

using namespace std;

```



```

// 定义状态结构体
struct State {
    int a, b;
    string op_desc; // 描述当前操作
    int parent_id; // 记录父节点下标以便回溯路径
};

void solveWaterJug(int capA, int capB, int L) {
    if (L > max(capA, capB)) {
        cout << "No Solution (目标水量超过了容器最大容量)" << endl;
        return;
    }
    vector<State> history; // 相当于队列的增强版, 保留所有节点用于回溯
    int head = 0; // 队列头指针, 模拟 BFS
    // 记录状态是否访问过
    vector<vector<bool>> visited(capA + 1, vector<bool>(capB + 1, false));
    history.push_back({0, 0, "初始状态", -1});
    visited[0][0] = true;
    int target_id = -1;
    while (head < history.size()) {
        State curr = history[head];
        if (curr.a == L || curr.b == L) {
            target_id = head;
            break;
        }
        // 定义 6 种可能的操作
        int next_states[6][2] = {
            {capA, curr.b}, // 1. 填满 A
            {curr.a, capB}, // 2. 填满 B
            {0, curr.b},    // 3. 倒空 A
            {curr.a, 0},    // 4. 倒空 B
            // 5. A 倒 B: 实际倒入量为 min(A 当前水, B 剩余容量)
            {curr.a - min(curr.a, capB - curr.b), curr.b + min(curr.a, capB - curr.b)},
            // 6. B 倒 A: 实际倒入量为 min(B 当前水, A 剩余容量)
            {curr.a + min(curr.b, capA - curr.a), curr.b - min(curr.b, capA - curr.a)}
        };
        string ops[6] = {
            "装满容器 1", "装满容器 2", "倒空容器 1", "倒空容器 2",
            "将容器 1 的水倒入容器 2", "将容器 2 的水倒入容器 1"
        };
        for (int i = 0; i < 6; i++) {
            int na = next_states[i][0];
            int nb = next_states[i][1];
            if (!visited[na][nb]) {

```

```

        visited[na][nb] = true;
        history.push_back({na, nb, ops[i], head});
    }
}
head++;
}
if (target_id == -1) {
    cout << "No Solution" << endl;
} else {
    // 回溯打印路径
    vector<int> path;
    int curr = target_id;
    while (curr != -1) {
        path.push_back(curr);
        curr = history[curr].parent_id;
    }

    cout << "找到解决方案, 步骤如下: " << endl;
    for (int i = path.size() - 1; i >= 0; i--) {
        int id = path[i];
        cout << "步骤 " << path.size() - 1 - i << ": "
            << history[id].op_desc << " => (容器 1: " << history[id].a
            << " 升, 容器 2: " << history[id].b << " 升)" << endl;
    }
}
}

int main() {
    cout << "=====倒水问题 =====" << endl;
    int capA = 3, capB = 5, target = 4; // 这里以 3 升、5 升杯子装 4 升水为例
    cout << "设定: 容器 1 容量=" << capA << ", 容器 2 容量=" << capB << ", 目标水量=" << target <<
endl;
    solveWaterJug(capA, capB, target);

    return 0;
}

```